

05. Ray Tracing in Vulkan

Objectives of this Lecture

- **Understanding the Stages of Ray Tracing with Vulkan**
- **Comprehending the Features Required for Ray Tracing**
 - Acceleration Structure
 - Types of Shaders (Ray Generation, Ray Hit, Ray Miss)
 - Shader Binding Table (SBT)
- **Example Code** 'Vulkan_ex01.cpp'



Window System Setup

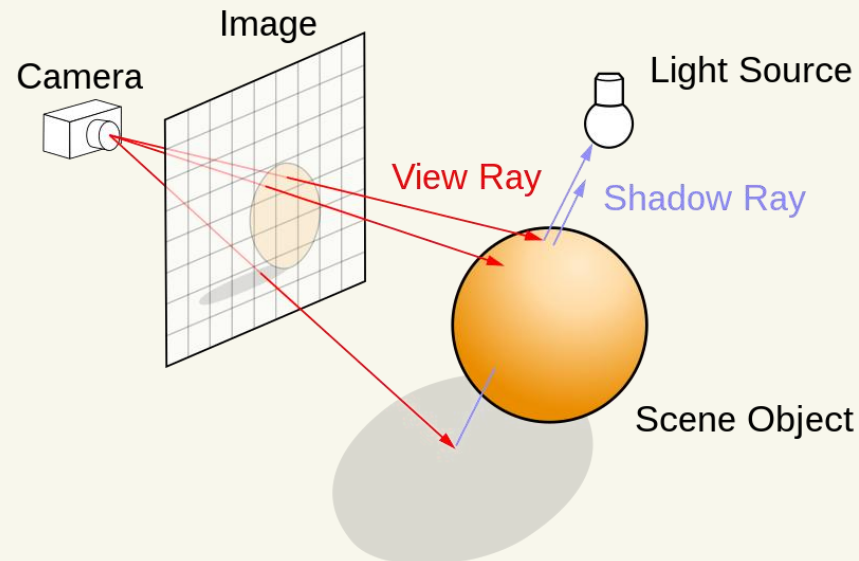
- GLFW
- Glm
- tiny_obj_loader.h

However, Vulkan currently does not provide ray tracing extension functionality for Mac OS

Ray Tracing Overview

Ray Tracing Algorithm Overview

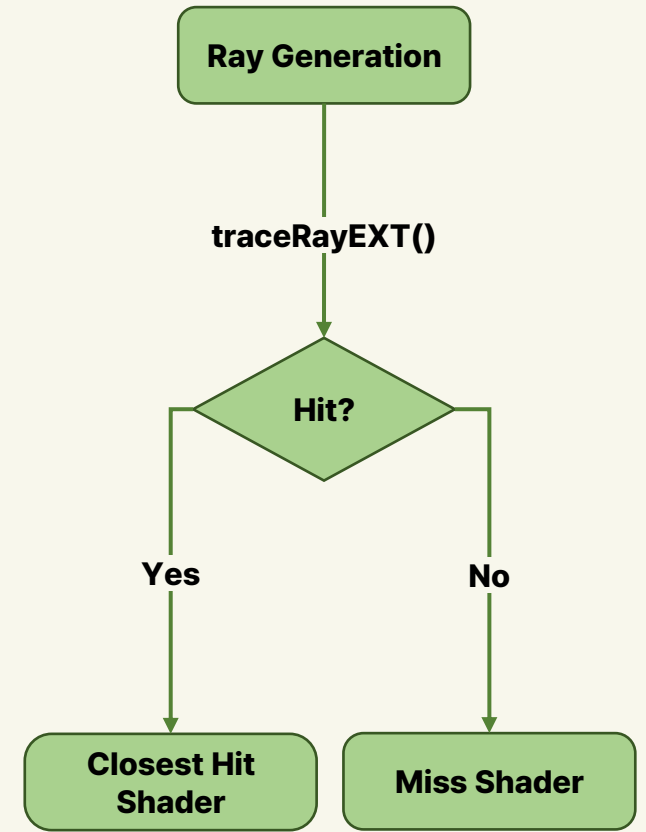
- Creates 3D visual images.
- Calculates pixel color by tracing rays from an imaginary eye through each pixel, considering object properties and light interaction.
- Enhances efficiency using backward tracing from the camera.



Composition of Ray Tracing

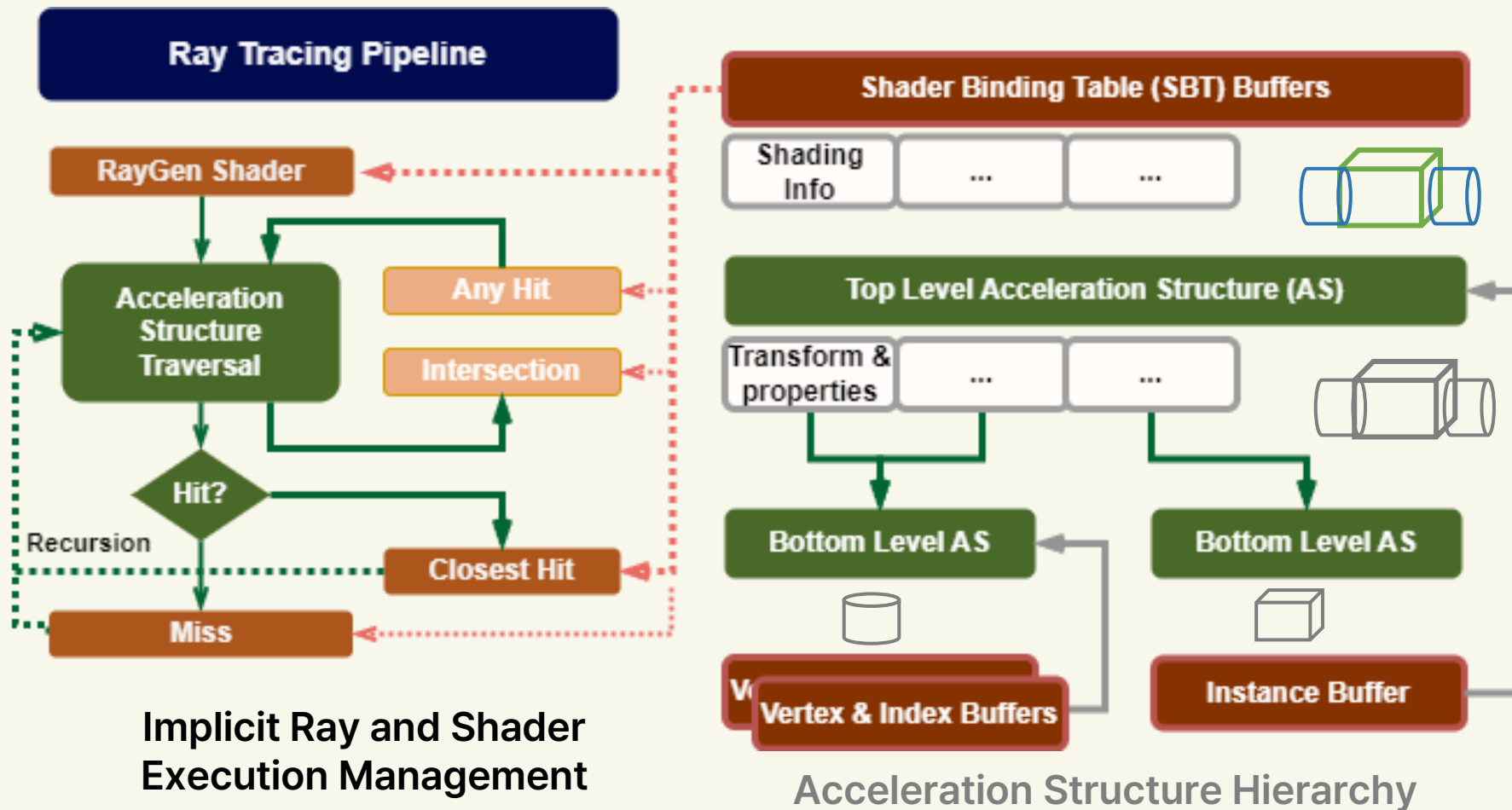
Ray Tracing

- **Ray Generation**
 - Compute ray origin, direction, and invoke shaders.
- **Intersect with Object**
 - Determine if a ray intersects any scene objects.
- **Calculate Shader**
 - Calculate color or shading based on intersections.

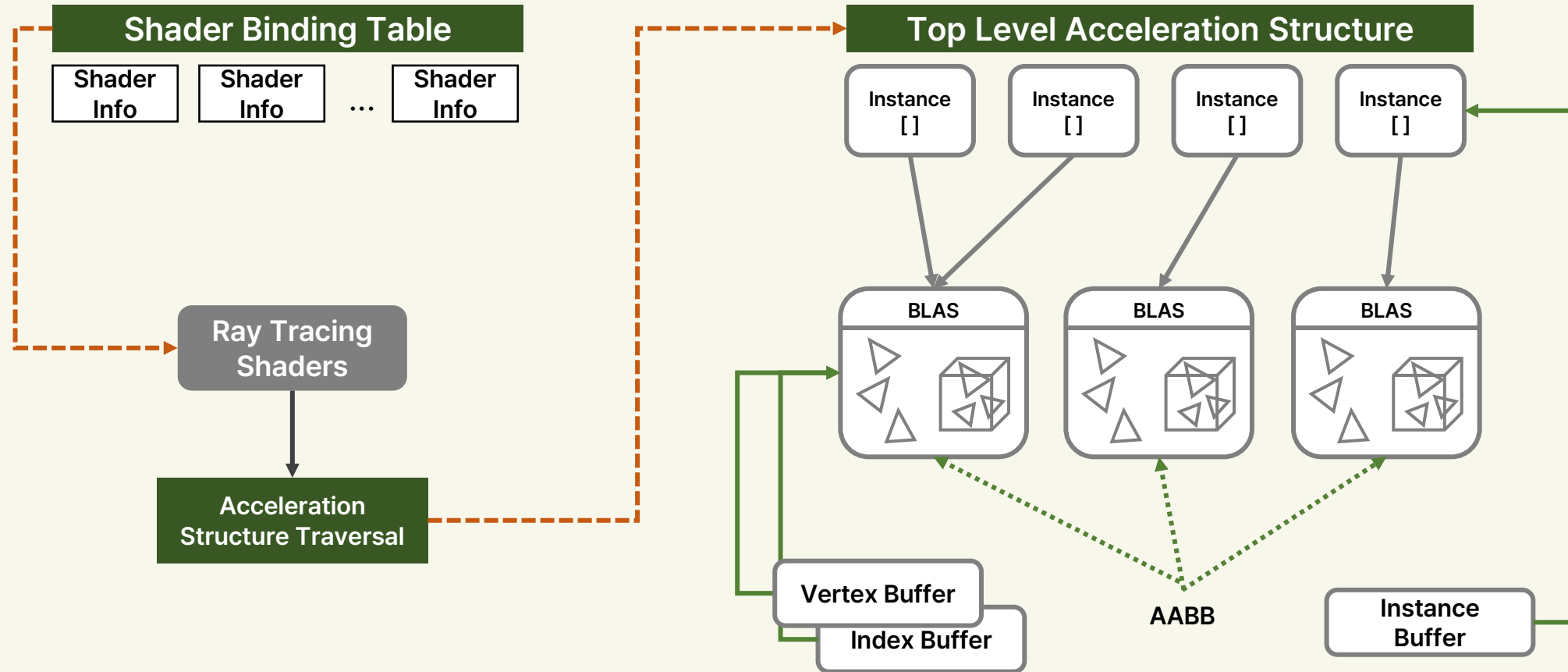


Composition of Ray Tracing

Implementation Steps for Vulkan Ray Tracing



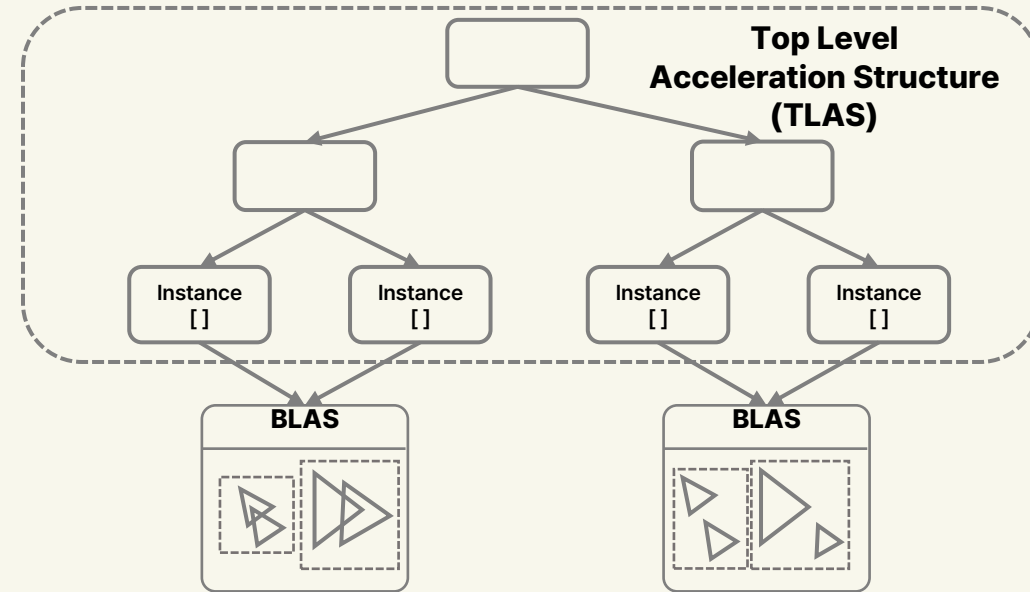
Acceleration Structures



Acceleration Structures

TLAS (Top Level Acceleration Structure)

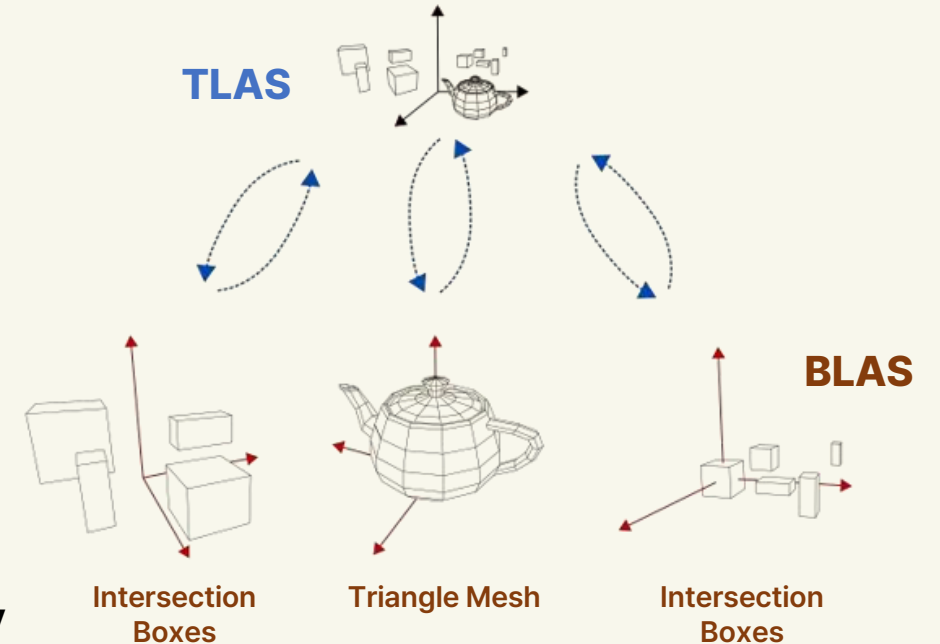
- Represents the entire scene in ray tracing
- Contains instances of BLAS, which hold geometric data
- Crucial for dynamic scenes, reflecting object movements and transformations
- Organizes the spatial arrangement of BLAS in the scene



Acceleration Structures

BLAS (Bottom Level Acceleration Structure)

- Stores detailed geometric data like vertices and triangles
- Used for individual or groups of static objects
- Pre-compiled for efficiency; unchanged for static geometry
- Optimized for static scenes or non-transforming objects



Building a Ray Tracing Engine in Vulkan

A Deep Dive into the RayTracedScene Implementation

- Vulkan 1.2 Ray Tracing Extensions
- Acceleration Structures (BLAS / TLAS)
- SBT (Shader Binding Table)
- RayGen / Miss / Closest Hit pipeline
- Real-time rendering pipeline using vkCmdTraceRaysKHR

Learning Objectives

- How Vulkan Ray Tracing is initialized and configured
- How geometry is processed into BLAS
- How scene instances compose the TLAS
- How descriptor sets & SBT connect shaders together
- How the rendering loop triggers ray tracing
- How your code implements a full real-time ray tracer

Project Overview

Main components

C++ side

- RayTracedScene class
- Geometry loading, BLAS/TLAS creation
- Descriptor sets, RT pipeline, SBT
- Command buffers, synchronization, render loop

Shader side

- raygen.rgen – generate rays and write pixels
- closesthit.rhit – color visible objects
- miss.rmiss – background color
- shadow.rmiss – handles shadow ray payload (extension point)

Project Overview

RayTracedScene

- |——— initWindow()
- |——— initVulkan()
 - |——— createInstance()
 - |——— pickPhysicalDevice()
 - |——— createLogicalDevice()
 - |——— createSwapChain()
 - |——— setupScene()
 - |——— createBottomLevelAS()
 - |——— createTopLevelAS()
 - |——— createRTPipeline()
 - |——— createShaderBindingTable()
- |——— mainLoop()
 - |——— drawFrame()
- |——— cleanup()

Vulkan Instance & Device

Required Instance Extensions

- Surface extensions from GLFW
- VK_EXT_debug_utils for validation (Debug Messenger)

Required Device Extensions

- VK_KHR_swapchain
- VK_KHR_acceleration_structure
- VK_KHR_ray_tracing_pipeline
- VK_KHR_deferred_host_operations
- VK_KHR_buffer_device_address
- VK_EXT_descriptor_indexing

Ray Tracing Feature Chain

- BufferDeviceAddress → AccelerationStructure → RayTracingPipeline → DescriptorIndexing

Swapchain & Storage Image

Swapchain

- Standard selection of format, present mode, extent
- Images used as final presentation targets

Storage Image

Used as ray tracing output buffer

- Format matches swapchain
- Usage: STORAGE | TRANSFER_SRC
- Written by RayGen shader
- Copied to swapchain image every frame

Scene Setup

ObjectInstance

- .obj model path
- Position / rotation / scale
- Base color (per-instance)

Light Structure

- Position
- Color
- Intensity
- Enabled flag

Example Scene

- 1× object ("PiggyBank.obj")
- 1× point light above the scene

Uniform Buffer Contents

UniformBufferObject

Contains:

- Inverse View matrix
- Inverse Projection matrix
- Camera world position
- Array of lights
- Light count

Updated Every Frame

- Camera transform
- Light data
- Projection inversion for ray direction reconstruction

Loading Geometry

Using tinyobjloader

- Reads vertices, normals, indices
- Deduplicates vertices using `unordered_map<Vertex,uint32_t>`
- Applies object scale

GPU Buffer Creation

Two buffers produced:

- Vertex buffer (device addressable)
- Index buffer (device addressable)
- Used later by BLAS builder.

Bottom Level AS (BLAS)

Purpose

Acceleration structure representing geometry of a single mesh.

Construction Steps

- Prepare `VkAccelerationStructureGeometryKHR` (triangles)
- Query required build sizes
- Create AS buffer
- Create scratch buffer
- Execute build using `vkCmdBuildAccelerationStructuresKHR`

Output

- One BLAS per object
- Stored inside `geometryDataList[]`

Top Level AS (TLAS)

Purpose

Represents the scene, containing multiple BLAS instances.

Each Instance Contains:

- 3D transform matrix (translate → rotate → scale)
- BLAS device address
- Custom instance index (used for shading)
- Visibility mask

Build Process

- Upload array of `VkAccelerationStructureInstanceKHR`
- Build TLAS similarly to BLAS
- Store result in `topLevelAS`

Instance Color Buffer

Per-Instance Data

- Stores RGBA color for each object
- Used in shaders through:

`gl_InstanceCustomIndexEXT`

Buffer Properties

- Storage buffer
- Host visible & coherent
- Used in closest-hit shader to color objects

Descriptor Set Layout

Bindings

Binding	Type	Usage
0	Acceleration Structure	TLAS
1	Storage Image	Ray tracing output
2	Uniform Buffer	Camera & lights
3	Storage Buffer	Instance colors

Matches what the RayGen and Hit shaders expect.

Ray Tracing Pipeline Setup

Shader Modules

- raygen.rgen.spv (Ray generation)
- miss.rmiss.spv (Primary miss)
- shadow.rmiss.spv (Shadow ray miss)
- closesthit.rchit.spv (Geometry shading)

Shader Groups

- RayGen Group
- Miss Group
- Shadow Miss Group
- Closest Hit Group

Pipeline Parameters

- Up to 2 recursion levels
- Uses the descriptor set layout created earlier

Ray Tracing Shader Roles

Ray Tracing Shader Stages

Ray Generation Shader (**raygen.rgen**)

- Launches one ray per pixel
- Converts pixel coordinates → world-space rays
- Calls a ray traversal function to query the scene

Closest Hit Shader (**closesthit.rchit**)

- Executes when a ray intersects geometry
- Computes surface response (material, lighting, shadows)
- Writes final result to the ray payload

Miss Shader (**miss.rmiss**)

- Runs when the ray does not hit anything
- Produces a background or fallback value

Shadow Miss Shader (**shadow.rmiss**)

- Runs for rays cast toward lights
- Determines whether a point is in shadow

Ray Tracing Shader Roles

Ray Generation Shader (raygen.rgen)

- Computes normalized device coordinates per pixel
- Converts them to world-space origin + direction
- Stores a constant color, without performing scene traversal

What should be added

1. Initialize a ray payload

- A vec3 payload for storing hit results
- Must be declared with layout(location = X)

2. Call a ray tracing function `traceRayEXT()`

Traverses the Top-Level Acceleration Structure

- Accepts parameters such as:
 - Acceleration structure handle
 - Ray flags (ex: opaque, no culling)
 - Visibility mask
 - SBT record indices (ray type)
 - Ray origin (world space)
 - Minimum / maximum distances
 - Ray direction (world space)
 - Payload index

Ray Tracing Shader Roles

Closest Hit Shader (closesthit.rchit)

Obtains per-instance color using:

`gl_InstanceCustomIndexEXT`

Computes a hit position using:

Ray origin + direction × hit distance

Writes only a flat object color to payload

What must add

1. Shading normal

Compute world-space normal using a helper function

Function uses `gl_ObjectToWorldEXT`

Must be normalized

2. Ambient term

Multiply base color by a small constant factor

3. Loop through lights

Iterate through `ubo.lightCount`

Check if each light is enabled

4. Light vector & attenuation

Compute direction from hit point → light position

Compute distance magnitude

5. Shadow ray

Fire a secondary ray using the same ray traversal function but:

Use different ray flags appropriate for shadow queries

Skip the closest hit shader

Terminate on first hit

Use a separate payload (boolean)

determine:

Ray origin offset

Max distance to light

Payload location

6. Lighting (computed only when not shadowed)

Diffuse: `dot(normal, lightDir)`

Specular: using camera direction + half vector

Combine contributions into final color

7. Write result to hit payload

Overwrite the vector payload with shaded result

Ray Tracing Shader Roles

Miss Shaders (miss.rmiss, shadow.rmiss)

General Miss Shader (miss.rmiss)

Runs when the primary ray does not hit geometry

Must write a background value into the payload

Shadow Miss Shader (shadow.rmiss)

Runs when a shadow ray reaches the light without hitting anything

Must modify a boolean payload

Shader Binding Table (SBT)

Contains Shader Records

- RayGen record
- 2× Miss records
- 1× Hit record

Built by:

- Querying shader group handles
- Storing them into 3 GPU buffers:
 - raygenShaderBindingTable
 - missShaderBindingTable
 - hitShaderBindingTable

Required for vkCmdTraceRaysKHR

Executing Ray Tracing

```
vkCmdTraceRaysKHR(  
    commandBuffer,  
    &raygenRegion,  
    &missRegion,  
    &hitRegion,  
    &callableRegion,  
    width, height, 1  
); After Tracing:
```

- Transition storage image → transfer-src
- Transition swapchain image → transfer-dst
- Copy ray-traced pixels into swapchain image
- Transition images to presentation and general use

Render Loop

Per Frame

- Acquire swapchain image
- Update UBO (camera & lights)
- Record command buffer
- Submit to graphics queue
- Present to screen

Double Buffering

- `MAX_FRAMES_IN_FLIGHT = 2`
- Avoids CPU/GPU stalls.

Swapchain Recreation

Occurs on:

- Window resize
- Surface lost or outdated

Steps:

- Wait for device idle
- Destroy image views & swapchain
- Recreate swapchain
- Recreate storage image
- Recreate descriptor sets

Cleanup

Destroy everything in reverse order:

- Frames & sync objects
- Command pool
- Descriptor sets & pools
- SBT buffers
- Ray tracing pipeline
- BLAS & TLAS
- Geometry buffers
- Swapchain resources
- Device, Instance
- GLFW window

Summary

What This Code Demonstrates

- A complete Vulkan ray tracing renderer
- Single-file architecture for learning clarity
- Full AS build pipeline (BLAS/TLAS)
- Descriptor & shader system integration
- Runtime ray tracing with SBT + pipelines
- Real-time rendering loop with swapchain management

Concepts Reinforced

- Vulkan memory model
- Device addresses
- Ray tracing shader stages
- Rendering synchronization